INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY, BHUBANESWAR

Department of Computer Science & Engineering | DAA Laboratory (CS3101)

LAB 10 REPORT: ADVANCED GRAPHS & DISJOINT SET UNION (DSU)

Student: Rishav Kumar **Roll:** B125099 **Branch:** B.Tech CSE (B1) **Instructor:** Dr. A. K. Dash **Date:** Oct 2026

Executive Summary & Paradigm Foundations: This laboratory investigates advanced graph representations and dynamic connectivity algorithms driven by the **Disjoint Set Union (DSU / Union-Find)** data structure across five fundamental problems. Using **Path Compression** and **Union by Rank**, DSU achieves an optimal amortised inverse Ackermann bound $\mathcal{O}(\alpha(V))$. The laboratory explores parity potentials for online 2-colorability and odd-cycle detection, 2D lattice coordinate flattening for dynamic island tracking, size-constrained component maximization with lazy priority queues, the offline time-reversal paradigm for vertex deletion dynamics, and symmetric group transposition generation for lexicographical permutation optimization.

Index of Lab 10 Problems & Complexity Summary

Q. No.	Problem Name	Algorithmic Paradigm	Time	Space	Status
Q1	Online Bipartiteness Testing	Parity Potential DSU & Cycle Trace	$\mathcal{O}(M \cdot \alpha(V))$	$\mathcal{O}(V+M)$	Verified
Q2	Dynamic Grid Islands (II)	2D Coordinate Flattening + DSU	$\mathcal{O}(k \cdot \alpha(RC))$	$\mathcal{O}(R \cdot C)$	Verified
Q3	Size-Constrained Max Weight	Augmented DSU + Lazy Max-Heap	$\mathcal{O}(M \log(V+M))$	$\mathcal{O}(V+M)$	Verified
Q4	Reverse Time-Step Deletion	Offline Time Reversal DSU	$\mathcal{O}((V+E)\alpha(V))$	$\mathcal{O}(V+E)$	Verified
Q5	Smallest Permutation Swaps	Symmetric Group Transpositions + DSU	$\mathcal{O}(M\alpha(n)+n\log n)$	$\mathcal{O}(n+M)$	Verified

Q1: Online Graph-Bipartiteness Testing

Parity Potential DSU · $\mathcal{O}(M \cdot \alpha(V))$

Problem Statement: An undirected graph receives edges online one by one. Maintain whether the graph remains bipartite (contains no odd-length cycles). If an edge (u, v) does not introduce an odd cycle, add it; otherwise, report the violation and skip it.

Mathematical Formulation & Parity Invariant: By König's Theorem, a graph is bipartite iff it is 2-colorable with colors $\{0, 1\}$. Each node x stores $p(x) \in \{0, 1\}$, the parity distance to its parent in the forest. During path compression, $P(x \to r) = \bigoplus p(w)$ along the branch. When edge (u, v) arrives: $r_u = \text{find}(u, P_u), \quad r_v = \text{find}(v, P_v)$.

- **If $r_u = r_v$:** Tree path length parity is $P_u \oplus P_v$. If $P_u = P_v$, adding (u, v) creates an **odd cycle** of length $\text{even} + 1 = \text{odd}$. The edge is **rejected**. If $P_u \neq P_v$, it forms an even cycle and is **accepted (redundant)**.
- **If $r_u \neq r_v$:** Vertices are in disjoint trees. The edge is **accepted**. We link r_u to r_v setting $p(r_u) = P_u \oplus P_v \oplus 1$.

Cycle Reconstruction: BFS on accepted tree edges reconstructs the exact odd cycle in $\mathcal{O}(V)$ time. **Complexity:** Time: $\mathcal{O}(M \cdot \alpha(V))$, Auxiliary Space: $\mathcal{O}(V + M)$.

Validation Trace ($V = 5, M = 6$):

Edge	Endpoints	Status	Reason / Cycle Detected
1	$(1, 2)$	Accepted	Cross-tree link: $P(1) = 1, P(2) = 0$
2	$(2, 3)$	Accepted	Cross-tree link: $P(3) = 1, P(2) = 0$
3	$(3, 4)$	Accepted	Cross-tree link: $P(4) = 0, P(3) = 1$
4	$(4, 5)$	Accepted	Cross-tree link: $P(5) = 1, P(4) = 0$
5	$(1, 5)$	Rejected	Odd Cycle: $1 \to 2 \to 3 \to 4 \to 5 \to 1$ (len 5)
6	$(2, 4)$	Rejected	Odd Cycle: $2 \to 3 \to 4 \to 2$ (len 3)

Final 2-Coloring Partition: Red: $\{2, 4\}$, Blue: $\{1, 3, 5\}$.

```
int findSet(BipartiteDSU *dsu, int x, int *p) {
    if (dsu->parent[x] == x) {
        *p = 0;
        return x;
    }
    int pParent = dsu->parity[x];
    int rootParity = 0;
    int root = findSet(dsu, dsu->parent[x],
&rootParity);
    dsu->parity[x] = pParent ^ rootParity;
    dsu->parent[x] = root;
    *p = dsu->parity[x];
    return root;
}

bool addEdgeOnline(BipartiteDSU *dsu, int u, int v) {
    int pu = 0, pv = 0;
    int ru = findSet(dsu, u, &pu);
    int rv = findSet(dsu, v, &pv);
    if (ru == rv) {
        if (pu == pv) return false; /* ODD CYCLE! */
        return true; /* Redundant even cycle */
    }
    int req = pu ^ pv ^ 1;
    if (dsu->rank[ru] < dsu->rank[rv]) {
        dsu->parent[ru] = rv;
        dsu->parity[ru] = req;
    } else {
        dsu->parent[rv] = ru;
        dsu->parity[rv] = req;
        if (dsu->rank[ru] == dsu->rank[rv]) dsu-
>rank[ru]++;
    }
    return true;
}
```

Q2: Dynamic Connected Components in a Grid (Number of Islands II)

Coordinate Flattening DSU · $\mathcal{O}(k \cdot \alpha(RC))$

Problem Statement: In an $R \times C$ grid initially filled with water, land cells are dynamically added one by one at $(r_1, c_1), ..., (r_k, c_k)$. Maintain and output the number of 4-directionally connected land components (islands) after each operation.

Coordinate Bijection & Component Invariants: We define a linear mapping $\varphi : [0, R-1] \times [0, C-1] \to [0, RC-1]$: $\varphi(r, c) = r \cdot C + c, \quad r = \lfloor \frac{\text{id}}{C} \rfloor, \quad c = \text{id} \bmod C$.

- When land is added at (r, c):
  1. If already land, island count remains unchanged.
  2. Otherwise, activate cell: $\text{grid}[r][c] = 1$, islands $\to$ islands $+ 1$.
  3. For each cardinal neighbor $(n_r, n_c) \in \mathcal{N}_4(r, c)$: If $\text{grid}[n_r][n_c] == 1$, let $u = \varphi(r, c), v = \varphi(n_r, n_c)$. If $\text{find}(u) \neq \text{find}(v)$: $\text{union}(u, v)$, islands $\to$ islands $- 1$.

Complexity: Inspecting 4 neighbors triggers at most 4 DSU operations per addition. Amortised time: $\mathcal{O}(RC + k \cdot \alpha(RC))$. Auxiliary space: $\Theta(RC)$.

Validation Trace (3×3 Grid, $k = 4$):

Step	Land Cell	Initial Count	Merges	Final Islands
1	$(1, 1)$	$0 + 1 = 1$	None	1
2	$(1, 2)$	$1 + 1 = 2$	Merged with $(1, 1)$	1
3	$(2, 3)$	$1 + 1 = 2$	None	2
4	$(2, 2)$	$2 + 1 = 3$	Merged $(1, 2)$ & $(2, 3)$	1

Island Count Sequence: $[1, 1, 2, 1]$.

```
int currId = r * cols + c;
if (grid[r][c] == 1) return islandCount;
grid[r][c] = 1;
dsu->parent[currId] = currId;
islandCount++;

int dr[4] = {-1, 1, 0, 0};
int dc[4] = {0, 0, -1, 1};
for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
        if (grid[nr][nc] == 1) {
            int neighborId = nr * cols + nc;
            int ru = findSet(dsu, currId);
            int rv = findSet(dsu, neighborId);
            if (ru != rv) {
                unionSets(dsu, ru, rv);
                islandCount--;
            }
        }
    }
}
```

Q3: Maximum Weight Component with Size Constraints

Augmented DSU + Lazy Max-Heap · $\mathcal{O}(M \log(V + M))$

Problem Statement: In a vertex-weighted graph with weights w_v, edges are added sequentially. Dynamically track the maximum total vertex weight among all connected components whose size does not exceed a threshold K.

Non-Monotonicity & Lazy Invalidation Strategy: The feasible component candidate set is $\mathcal{F} = \{C \in \mathcal{C} \text{ mid } |C| \leq K\}$. When edge (u, v) merges C_u and C_v: $|C_{\text{new}}| = |C_u| + |C_v|, \quad W(C_{\text{new}}) = W(C_u) + W(C_v)$. If $|C_{\text{new}}| > K$, the merged component becomes **ineligible**, potentially decreasing W^*.

- We augment each DSU root with `compSize`, `compWeight`, and a `version` counter.
- Valid merged roots with $|C| \leq K$ are pushed into a binary Max-Heap.
- During query time, stale heap entries (outdated version, non-representative, or size $> K$) are lazily discarded.

Complexity: DSU runs in $\mathcal{O}(\alpha(V))$. Each edge pushes at most 1 heap node in $\mathcal{O}(\log|H|)$. Total time: $\mathcal{O}(V + M \log(V + M))$. Auxiliary space: $\Theta(V + M)$.

Validation Trace ($V = 5, K = 3$, weights: [10, 20, 15, 30, 25]):

Step	Edge	Merged Components	New Comp	Max (Size ≤ 3)
Init	None	5 singletons	All size 1	30 (Vertex 4)
1	$(1, 2)$	$\{1\}$ (10) + $\{2\}$ (20)	Size 2, wt 30	30 (Vertices 4, 1)
2	$(3, 4)$	$\{3\}$ (15) + $\{4\}$ (30)	Size 2, wt 45	45 (Root 3)
3	$(1, 3)$	$\{1, 2\}$ (30) + $\{3, 4\}$ (45)	Size 4 (> 3!)	25 (Vertex 5)
4	$(4, 5)$	$\{1..4\}$ (75) + $\{5\}$ (25)	Size 5 (> 3!)	NONE (All > 3)

```
bool getMaxValidWeight(MaxHeap *heap, ConstrainedDSU *dsu,
                       int K, long long *maxVal, int *bestRoot) {
    while (heap->size > 0) {
        HeapNode top = heap->data[0];
        int r = top.rootId;
        if (dsu->parent[r] == r &&
            dsu->version[r] == top.version &&
            dsu->compSize[r] <= K) {
            *maxVal = top.totalWeight;
            *bestRoot = r;
            return true;
        }
        /* Discard stale or invalidated entry */
        heap->data[0] = heap->data[--heap->size];
        maxHeapify(heap, 0);
    }
    return false;
}
```

Q4: Reverse Time-Step DSU Application

Offline Time Reversal · $\mathcal{O}((V+E)\alpha(V))$

Problem Statement: Vertices are sequentially deleted from a vertex-weighted undirected graph in order $P = [p_1, ..., p_V]$. After each deletion, compute the maximum sum of vertex values among all remaining connected components.

Time-Reversal Duality: Online deletion splits components, requiring complex dynamic connectivity ($\mathcal{O}(\log^2 V)$). Inverting the sequence backwards ($\tau = V - t + 1$) transforms destructive vertex removals into **incremental vertex additions**:

- Reverse Step τ: Activate vertex $u = p_{\{V-\tau+1\}}$.
- For each neighbor v of u with $\text{isActive}[v] == 1$, execute DSU union and aggregate component weights: $W(\text{newRoot}) = W(r_u) + W(r_v)$.
- Maintain running maximum: $\text{curMax} = \max(\text{curMax}, W(\text{root}(u)))$.
- Map result: $\text{ans}[\text{step} - 1] = \text{curMax}$, with $\text{ans}[V - 1] = 0$.

Complexity: Building adjacency list: $\mathcal{O}(V + E)$. DSU operations: $\mathcal{O}((V + E)\alpha(V))$. Total space: $\Theta(V + E)$.

Validation Trace (Path $1 - 2 - 3 - 4$, weights $[1, 2, 3, 4]$, $P = [2, 4, 1, 3]$):

Deletion	Vertex Deleted	Remaining Vertices	Max Component Weight
Step 1	Vertex 2	$\{1\}, \{3, 4\}$	7 (from $\{3, 4\}$)
Step 2	Vertex 4	$\{1\}, \{3\}$	3 (from $\{3\}$)
Step 3	Vertex 1	$\{3\}$	3 (from $\{3\}$)
Step 4	Vertex 3	$\varnothing$ (Empty graph)	0

```
for (int step = v - 1; step >= 0; step--) {
    int u = deletionOrder[step] - 1;
    isActive[u] = true;
    if (dsu->compWeight[u] > currentMax)
        currentMax = dsu->compWeight[u];

    for (AdjNode *curr = g->head[u]; curr; curr = curr->next) {
        int nb = curr->to;
        if (isActive[nb]) {
            int ru = findSet(dsu, u);
            int rv = findSet(dsu, nb);
            if (ru != rv) {
                unionSets(dsu, ru, rv);
                int mr = findSet(dsu, u);
                if (dsu->compWeight[mr] > currentMax)
                    currentMax = dsu->compWeight[mr];
            }
        }
    }
    if (step > 0) maxSumAfterDeletion[step - 1] = currentMax;
}
maxSumAfterDeletion[v - 1] = 0;
```

Q5: Lexicographically Smallest Connected Component Permutation

Symmetric Group Transpositions · $\mathcal{O}(M\alpha(n) + n \log n)$

Problem Statement: Given a permutation A of numbers $1...n$ and allowable pairwise swap pairs (u, v) forming edges in an undirected graph, determine the lexicographically smallest permutation achievable through any number of valid swaps.

Symmetric Group Invariant & Greedy Ordering: Each allowable swap (u, v) is a transposition generator. By the transposition generation theorem in abstract algebra, transpositions corresponding to edges in any connected graph generate the entire symmetric group $S_{\{|C|\}}$.

- Hence, elements occupying indices in the same connected component C can be rearranged into **any** arbitrary permutation.
- To achieve lexicographical minimality, for each connected component:
  1. Extract the sorted list of index positions: $i_1 < i_2 < ... < i_k$.
  2. Extract values $A[i_1], ..., A[i_k]$ and sort them in ascending order: $v_1 \leq v_2 \leq ... \leq v_k$.
  3. Assign $A^*[i_r] = v_r$ for all $1 \leq r \leq k$.

Complexity: DSU unions take $\mathcal{O}(M\alpha(n))$. Sorting disjoint components takes $\sum |C| \log |C| \leq \mathcal{O}(n \log n)$. Total space: $\Theta(n + M)$.

Validation Trace ($n = 5, A = [5, 3, 1, 4, 2]$, Swaps: $(1, 3), (2, 5), (3, 5)$):

Component	Indices	Initial Values	Sorted Assignment
Root 4	$\{4\}$	$[4]$	$A[4] = 4$
Root 5	$\{1, 2, 3, 5\}$	$[5, 3, 1, 2]$	$A[1] = 1, A[2] = 2, A[3] = 3, A[5] = 5$

Final Result: $[1, 2, 3, 4, 5]$ (Sorted optimal permutation).

```
void solveLexicographicallySmallest(int n, int *arr, int m,
                                    SwapPair *swaps, int *res) {
    PermDSU *dsu = createPermDSU(n);
    for (int i = 0; i < m; i++)
        unionSets(dsu, swaps[i].u - 1, swaps[i].v - 1);

    /* Group indices by component root */
    for (int r = 0; r < n; r++) {
        if (compSizes[r] > 0) {
            int sz = compSizes[r];
            int *vals = malloc(sz * sizeof(int));
            for (int k = 0; k < sz; k++)
                vals[k] = arr[compIndices[r][k]];

            qsort(vals, sz, sizeof(int), compareInts);

            for (int k = 0; k < sz; k++)
                res[compIndices[r][k]] = vals[k];
            free(vals);
        }
    }
}
```

Laboratory Synthesis & Complexity Verification: Across all 5 graph problems, Disjoint Set Union (DSU) delivers near-linear asymptotic complexity via Path Compression and Union by Rank ($\mathcal{O}(\alpha(N))$ amortised per query). Augmented potentials enable online parity tracking for bipartiteness (Q1), coordinate flattening maps 2D planar meshes into 1D addressable arrays (Q2), lazy priority queue integration solves non-monotonic constrained optimization (Q3), offline time-reversal circumvents intractable dynamic graph splitting (Q4), and algebraic symmetric group transpositions enable optimal lexicographical reconstruction (Q5). All algorithms were verified against edge cases and proved mathematically sound.